
Text-to-Graph Figure Generation via Structured Parsing and Iterative TikZ Refinement

Chang Wu
2024010783

An Yan
2024010836

Ce Yang
2024010781

TL;DR

We propose a training-free system that converts natural-language descriptions of small graphs into structured graph representations and then generates corresponding TikZ figures through code generation, rendering, and iterative validation.

1 Introduction

Graphs are widely used to represent relationships between entities, such as social networks, dependency structures, state transitions, and conceptual maps. However, manually drawing graphs for reports, papers, slides, or teaching materials can be time-consuming, especially when the graph structure is first described in natural language. A convenient text-to-figure system would allow users to describe nodes and edges in ordinary language and automatically obtain a clean graph visualization.

Existing graph drawing systems such as Graphviz provide powerful automatic layout algorithms for structured graph specifications [1]. TikZ and PGF are also widely used for producing high-quality figures in LaTeX documents [2]. However, these tools typically require users to write structured code or formal graph specifications. Recent progress in large language models has shown strong ability in text understanding and code generation [3, 4], which suggests that natural-language graph descriptions can potentially be converted into executable figure-generation code.

In this project, we aim to build a lightweight pipeline that takes as input a natural-language description of a graph, extracts a structured representation of nodes and edges, converts this representation into TikZ code, renders the code into a figure, and iteratively checks whether the generated figure matches the intended graph. Since our available data will likely consist only of a small number of graph-description examples, we do not plan to train a model. Instead, we will focus on prompt design, rule-based validation, structured intermediate representations, and iterative refinement.

2 Goal of Achievement

The main goal of this project is to demonstrate a working prototype that can generate graph figures from natural-language descriptions of small graphs. The expected input is a paragraph such as: “There are four nodes A, B, C, and D. A is connected to B and C. C points to D.” The expected output is a compiled graph figure, generated through TikZ code.

More specifically, our goals are:

- Build a pipeline that extracts a structured graph description, including nodes, edges, labels, directionality, and optional layout hints.
- Generate valid TikZ code from the structured description and compile it into a graph image.
- Implement automatic checks for common errors, such as missing nodes, missing edges, duplicated edges, invalid TikZ syntax, or ambiguous input descriptions.

- Add an iterative refinement mechanism that improves the structured representation or TikZ code when validation fails.
- Evaluate the system on a small manually collected benchmark of graph descriptions and measure whether the generated figure contains the correct nodes and edges.

We consider the project successful if the final demo can correctly generate figures for most simple graph descriptions involving approximately 3–10 nodes, directed or undirected edges, and basic node labels. The system should also be able to report uncertainty when the input description is ambiguous.

3 Methods

Our system will consist of five main stages: text preprocessing, structured graph extraction, TikZ code generation, rendering, and validation/refinement.

3.1 Text Preprocessing

The input will be a natural-language description of a graph. We will first normalize the text by removing unnecessary punctuation, splitting long descriptions into smaller clauses, and identifying possible graph-related expressions such as node declarations, edge declarations, direction words, and layout hints. For example, phrases such as “A connects to B”, “there is an edge from A to B”, and “A points to B” will be mapped to candidate edge relations.

3.2 Structured Graph Extraction

The preprocessed text will be converted into a structured intermediate representation. We plan to use a JSON-like schema, for example:

```
{
  "nodes": [
    {"id": "A", "label": "A"},
    {"id": "B", "label": "B"}
  ],
  "edges": [
    {"source": "A", "target": "B", "directed": true}
  ],
  "layout_hints": {
    "style": "circular"
  }
}
```

This structured representation is important because it separates semantic understanding from figure rendering. It also makes validation easier: before generating TikZ code, we can check whether every edge endpoint appears in the node list, whether duplicate nodes exist, and whether the graph description is internally consistent.

Because we do not expect to have enough data for training, the extraction step will be implemented using carefully designed prompts and lightweight rule-based post-processing. The language model will be asked to output only the structured schema, and the post-processing module will repair minor formatting errors when possible.

3.3 TikZ Code Generation

After obtaining the structured graph, the system will generate TikZ code. We will support a small set of layout strategies, such as circular layout, layered layout, and manually hinted layout. For simple graphs, node positions can be assigned using deterministic rules. For example, if no layout hint is provided, the system may place nodes evenly on a circle. If the input contains phrases such as “A is on the left of B”, this information can be converted into approximate coordinates.

The generated TikZ code will include node definitions, edge drawing commands, arrow styles for directed edges, and basic visual styles. A typical output may contain a `tikzpicture` environment with commands for placing nodes and drawing edges.

3.4 Rendering and Error Detection

The TikZ code will be compiled into a PDF or image. If compilation fails, the system will collect the compiler error message and use it to revise the TikZ code. If compilation succeeds, the system will still run structural checks by comparing the original structured graph with the generated code. For example, it will check whether all expected nodes appear in the code and whether every expected edge has a corresponding drawing command.

Optionally, if time permits, we will also explore image-level validation. The rendered graph image can be inspected using simple computer vision or a multimodal model to check whether the visible nodes and edges match the structured description. This would allow the system to detect cases where the code compiles but the resulting layout is visually unclear.

3.5 Iterative Refinement

When the system detects an error or ambiguity, it will enter an iterative refinement loop. The loop may revise either the structured representation or the TikZ code depending on the error type. For example, if the input says “connect A, B, and C in a chain” and the extracted structure is ambiguous, the system may ask whether the intended edges are $A - B$ and $B - C$ or all pairwise connections. If the TikZ code fails to compile, the system will revise the code based on the compiler message. To avoid infinite loops, the refinement process will stop after a fixed maximum number of iterations.

3.6 Evaluation

We will create a small evaluation set consisting of natural-language graph descriptions and manually written reference graph structures. Since the project is not focused on model training, this dataset will be used only for testing and demonstration. We will evaluate the system using the following metrics:

- **Node accuracy:** whether the generated graph contains the correct set of nodes.
- **Edge accuracy:** whether the generated graph contains the correct set of edges.
- **Direction accuracy:** whether directed edges have the correct direction.
- **Compilation success rate:** whether the generated TikZ code compiles successfully.
- **Human visual quality rating:** whether the final graph is readable and aesthetically acceptable.

4 Timeline

Period	Milestone	Expected Output
Week 11	Problem definition and example collection	Small set of graph-description examples
Week 12	Text-to-structure module	Initial parser for nodes, edges, and directionality
Week 13	Structure-to-TikZ module	TikZ code generator and basic graph layouts
Week 14	Rendering and refinement loop	Compilation checking and automatic revision
Week 15	Evaluation and poster preparation	Results, poster and demonstration

5 Workload Distribution

The workload will be divided into three main parts.

Chang Wu will be responsible for overall system design, integration, and project management. This includes defining the intermediate graph schema, coordinating the interface between modules, maintaining the final pipeline, and preparing the final demonstration.

An Yan will focus on natural-language processing and structured graph extraction. This includes designing prompts or rule-based extraction methods, handling ambiguous descriptions, normalizing node and edge expressions, and building the small evaluation set.

Ce Yang will focus on TikZ code generation, rendering, and validation. This includes generating valid TikZ code from the structured graph, implementing layout strategies, detecting compilation errors, and designing the iterative refinement mechanism.

All team members will contribute to testing, error analysis, writing the final report, and preparing the poster presentation. We will meet regularly to evaluate examples, identify failure cases, and decide which features should be prioritized.

References

- [1] Emden R. Gansner and Stephen C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, 2000.
- [2] Till Tantau. *The TikZ and PGF Packages*, 2013. Manual for version 3.0.0.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [4] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.